

HW 09 - Modeling the GSS

Max Asmar

In this assignment we continue our exploration of the 2016 GSS dataset from the previous homework.

Getting started

Open the `hw-09-modeling-gss.rmd` file in your RStudio Notebook on JupyterHub. Knit the document to make sure it compiles without errors.

Learning Objectives

In this assignment, you will practice:

- Building predictive models using the **tidymodels** framework
- Creating recipes with preprocessing steps
- Specifying model types and engines
- Using workflows to combine recipes and model specifications
- Performing k-fold cross-validation to assess model performance
- Comparing models using ROC curves and accuracy metrics
- Calculating bootstrap confidence intervals for proportions

This assignment builds on concepts from lecture and introduces the **tidymodels** workflow for systematic model building.

Warm up

Before we introduce the data, let's warm up with some simple exercises. Update the YAML of your R Markdown file with your information, knit, commit, and push your changes. Make sure to commit with a meaningful commit message. Then, go to your repo on GitHub and confirm that your changes are visible in your Rmd **and** md files. If anything is missing, commit and push again.

Packages

We'll use the **tidyverse** package for much of the data wrangling and visualisation, the **tidymodels** package for modeling and inference, and the data lives in the **dsbox** package. These packages are already installed for you. You can load them by running the following in your Console:



Figure 1: Photo Mauro Mora on Unsplash

```
library(tidyverse)
library(tidymodels)
library(dsbox)
```

Data

The data can be found in the **dsbox** package, and it's called **gss16**. Since the dataset is distributed with the package, we don't need to load it separately; it becomes available to us when we load the package. You can find out more about the dataset by inspecting its documentation, which you can access by running `?gss16` in the Console or using the Help menu in RStudio to search for **gss16**. You can also find this information [here](#).

The tidymodels Workflow

This assignment uses the **tidymodels** framework for building predictive models. Here's the general workflow:

1. **Data splitting:** Divide data into training and testing sets
 - `initial_split()` - Create the split
 - `training()` and `testing()` - Extract the sets
2. **Recipe:** Define preprocessing steps
 - `recipe(outcome ~ predictors, data)` - Specify the model formula
 - `step_*()` functions - Add preprocessing steps (e.g., `step_dummy()`, `step_other()`)
3. **Model specification:** Define the model type
 - `logistic_reg()` - Specify logistic regression
 - `set_engine()` - Choose the computational engine
4. **Workflow:** Combine recipe and model
 - `workflow()` - Create workflow
 - `add_recipe()` and `add_model()` - Add components
5. **Cross-validation:** Assess model performance
 - `vfold_cv()` - Create folds
 - `fit_resamples()` - Fit model to each fold
 - `collect_metrics()` - Get performance metrics
6. **Final fit and prediction:** Fit to training, predict on testing
 - `fit()` - Train the final model
 - `predict()` - Make predictions
 - `roc_curve()` and `roc_auc()` - Evaluate predictions

Exercises

Scientific research

In this section we're going to build a model to predict whether someone agrees or doesn't agree with the following statement:

Even if it brings no immediate benefits, scientific research that advances the frontiers of knowledge is necessary and should be supported by the federal government.

The responses to the question on the GSS about this statement are in the `advfront` variable.

It's important that you don't recode the NAs, just the remaining levels.

1. Re-level the `advfront` variable such that it has two levels: `Strongly agree` and `"Agree"` combined into a new level called `agree` and the remaining levels (except NAs) combined into `"Not agree"`. Then, re-order the levels in the following order: `"Agree"` and `"Not agree"`. Finally, `count()` how many times each new level appears in the `advfront` variable.

```
gss16 <- gss16 %>%
  mutate(advfront = case_when(
    advfront %in% c("Strongly agree", "Agree") ~ "Agree",
    is.na(advfront) ~ NA_character_,
    TRUE ~ "Not agree" # All other cases
  ))

# Reorder levels
gss16 <- gss16 %>%
  mutate(advfront = factor(advfront, levels = c("Agree", "Not agree")))

# Count
gss16 %>% count(advfront)

## # A tibble: 3 x 2
##   advfront      n
##   <fct>      <int>
## 1 Agree      1159
## 2 Not agree   226
## 3 <NA>       1482
```

2. Combine the levels of the `polviews` variable such that levels that have the word "liberal" in them are lumped into a level called `"Liberal"` and those that have the word conservative in them are lumped into a level called `"Conservative"`. Then, re-order

You can do this in various ways. One option is to use the `str_detect()` function to detect the existence of words like liberal or conservative. Note that these sometimes show up with lowercase first letters and sometimes with upper case first letters. To detect either in the `str_detect()` function, you can use `"[L]iberal"` and `"[C]onservative"`. But feel free to solve the problem however you like, this is just one option!

the levels in the following order: "Conservative", "Moderate", and "Liberal". Finally, `count()` how many times each new level appears in the `polviews` variable.

```
gss16 <- gss16 %>%
  mutate(
    polviews = case_when(
      str_detect(polviews, "[Ll]iberal") ~ "Liberal",
      str_detect(polviews, "[Cc]onservative") ~ "Conservative",
      str_detect(polviews, "Moderate") ~ "Moderate"
    )
  ) %>%
  mutate(polviews = factor(polviews, levels = c("Conservative", "Moderate", "Liberal")))

gss16 %>%
  count(polviews)

## # A tibble: 4 x 2
##   polviews      n
##   <fct>      <int>
## 1 Conservative  928
## 2 Moderate    1032
## 3 Liberal      796
## 4 <NA>        111
```

There are 928 Conservative respondents, 1032 moderate respondents, and 796 liberal respondents in the data.

3. Create a new data frame called `gss16_advfront` that includes the variables `advfront`, `educ`, `polviews`, and `wrkstat`. Then, use the `drop_na()` function to remove rows that contain NAs from this new data frame. Sample code is provided below.

```
gss16_advfront <- gss16 %>%
  select(advfront, educ, polviews, wrkstat) %>%
  drop_na()
```

4. Split the data into training (75%) and testing (25%) data sets. Make sure to set a seed before you do the `initial_split()`. Call the training data `gss16_train` and the testing data `gss16_test`. Sample code is provided below. Use these specific names to make it easier to follow the rest of the instructions.

```
set.seed(123)
gss16_split <- initial_split(gss16_advfront)
gss16_train <- training(gss16_split)
gss16_test  <- testing(gss16_split)
```

Checkpoint - Verify Your Data Preparation:

Before building models, check that your data setup is correct:

```
# Check advfront has 2 levels (plus NAs)
gss16 %>% count(advfront)

## # A tibble: 3 x 2
##   advfront      n
##   <fct>      <int>
## 1 Agree      1159
## 2 Not agree   226
## 3 <NA>       1482

# Check polviews has 3 levels (plus NAs)
gss16 %>% count(polviews)

## # A tibble: 4 x 2
##   polviews      n
##   <fct>      <int>
## 1 Conservative  928
## 2 Moderate     1032
## 3 Liberal       796
## 4 <NA>         111

# Check your training/testing split
nrow(gss16_train) # Should be ~75% of gss16_advfront

## [1] 996

nrow(gss16_test)  # Should be ~25% of gss16_advfront

## [1] 332

# Check no NAs in your analysis dataset
gss16_advfront %>% summarise(across(everything(), ~sum(is.na(.))))

## # A tibble: 1 x 4
##   advfront educ polviews wrkstat
##   <int> <int>   <int>   <int>
## 1      0    0       0       0
```

All NA counts should be 0 for `gss16_advfront`. If not, revisit Question 3.

-
5. Create a recipe with the following steps for predicting `advfront` from `polviews`, `wrkstat`, and `educ`. Name this recipe `gss16_rec_1`. (We'll create one more recipe later, that's why we're naming this recipe `_1`.) Sample code is provided below.

- `step_other()` to pool values that occur less than 10% of the time (`threshold = 0.10`) in the `wrkstat` variable into "Other".
- `step_dummy()` to create dummy variables for `all_nominal()` variables that are predictors, i.e. `all_predictors()`

```
gss16_rec_1 <- recipe(advfront ~ polviews + wrkstat + educ, data = gss16_train) %>%
  step_other(wrkstat, threshold = 0.10, other = "Other") %>%
  step_dummy(all_nominal(), -all_outcomes())
```

6. Specify a logistic regression model using "glm" as the engine. Name this specification `gss16_spec`. Sample code is provided below.

```
gss16_spec <- logistic_reg() %>%
  set_engine("glm")
```

7. Build a workflow that uses the recipe you defined (`gss16_rec`) and the model you specified (`gss16_spec`). Name this workflow `gss16_wflow_1`. Sample code is provided below.

```
gss16_wflow_1 <- workflow() %>%
  add_model(gss16_spec) %>%
  add_recipe(gss16_rec_1)
```

8. Perform 5-fold cross validation. specifically,

- split the training data into 5 folds (don't forget to set a seed first!),
- apply the workflow you defined earlier to the folds with `fit_resamples()`, and
- `collect_metrics()` and comment on the consistency of metrics across folds (you can get the area under the ROC curve and the accuracy for each fold by setting `summarize = FALSE` in `collect_metrics()`)
- report the average area under the ROC curve and the accuracy for all cross validation folds `collect_metrics()`

```
set.seed(156)
gss16_folds <- vfold_cv(gss16_train, v = 5)

gss16_fit_rs_1 <- gss16_wflow_1 %>%
  fit_resamples(gss16_folds)

collect_metrics(gss16_fit_rs_1, summarize = FALSE)
```

```
## # A tibble: 15 x 5
##   id    .metric    .estimator .estimate .config
##   <chr> <chr>      <chr>      <dbl> <chr>
## 1 Fold1 accuracy    binary      0.805 Preprocessor1_Model1
## 2 Fold1 roc_auc     binary      0.606 Preprocessor1_Model1
## 3 Fold1 brier_class binary      0.150 Preprocessor1_Model1
## 4 Fold2 accuracy    binary      0.799 Preprocessor1_Model1
## 5 Fold2 roc_auc     binary      0.543 Preprocessor1_Model1
## 6 Fold2 brier_class binary      0.163 Preprocessor1_Model1
## 7 Fold3 accuracy    binary      0.844 Preprocessor1_Model1
## 8 Fold3 roc_auc     binary      0.692 Preprocessor1_Model1
## 9 Fold3 brier_class binary      0.125 Preprocessor1_Model1
## 10 Fold4 accuracy    binary      0.874 Preprocessor1_Model1
## 11 Fold4 roc_auc     binary      0.623 Preprocessor1_Model1
## 12 Fold4 brier_class binary      0.111 Preprocessor1_Model1
## 13 Fold5 accuracy    binary      0.844 Preprocessor1_Model1
## 14 Fold5 roc_auc     binary      0.605 Preprocessor1_Model1
## 15 Fold5 brier_class binary      0.129 Preprocessor1_Model1
```

```
collect_metrics(gss16_fit_rs_1)
```

```
## # A tibble: 3 x 6
##   .metric    .estimator mean      n std_err .config
##   <chr>      <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy    binary      0.833     5 0.0140 Preprocessor1_Model1
## 2 brier_class binary      0.136     5 0.00936 Preprocessor1_Model1
## 3 roc_auc     binary      0.614     5 0.0237 Preprocessor1_Model1
```

The average area under the ROC curve is 0.614 and the accuracy for all cross validation folds is 0.833.

The AUC of ROC varied greatly between cross validation folds. For example, in fold 2, the AUC was 0.543, while in fold 3, the AUC was 0.692, which is 24% greater than the AUC of fold 2. This is relatively inconsistent.

9. Now, try a different, simpler model: predict `advfront` from only `polviews` and `educ`. Specifically,

- update the recipe to reflect this simpler model specification (and name it `gss16_rec_2`),

```
gss16_rec_2 <- recipe(advfront ~ polviews + educ, data = gss16_train) %>%
  step_dummy(all_nominal(), -all_outcomes())
```

- redefine the workflow with the new recipe (and name this new workflow ``gss16_wflow_2``),

```

gss16_wflow_2 <- workflow() %>%
  add_model(gss16_spec) %>%
  add_recipe(gss16_rec_2)

- perform cross validation, and

set.seed(156)

gss16_fit_rs_2 <- gss16_wflow_2 %>%
  fit_resamples(gss16_folds)

collect_metrics(gss16_fit_rs_2, summarize = FALSE)

## # A tibble: 15 x 5
##   id      .metric      .estimator .estimate .config
##   <chr> <chr>         <chr>         <dbl> <chr>
## 1 Fold1 accuracy      binary          0.81  Preprocessor1_Model1
## 2 Fold1 roc_auc        binary          0.623 Preprocessor1_Model1
## 3 Fold1 brier_class     binary          0.149 Preprocessor1_Model1
## 4 Fold2 accuracy      binary          0.799 Preprocessor1_Model1
## 5 Fold2 roc_auc        binary          0.527 Preprocessor1_Model1
## 6 Fold2 brier_class     binary          0.165 Preprocessor1_Model1
## 7 Fold3 accuracy      binary          0.844 Preprocessor1_Model1
## 8 Fold3 roc_auc        binary          0.706 Preprocessor1_Model1
## 9 Fold3 brier_class     binary          0.125 Preprocessor1_Model1
## 10 Fold4 accuracy      binary          0.874 Preprocessor1_Model1
## 11 Fold4 roc_auc        binary          0.602 Preprocessor1_Model1
## 12 Fold4 brier_class     binary          0.112 Preprocessor1_Model1
## 13 Fold5 accuracy      binary          0.844 Preprocessor1_Model1
## 14 Fold5 roc_auc        binary          0.602 Preprocessor1_Model1
## 15 Fold5 brier_class     binary          0.130 Preprocessor1_Model1

collect_metrics(gss16_fit_rs_2)

## # A tibble: 3 x 6
##   .metric      .estimator mean      n std_err .config
##   <chr>         <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy      binary    0.834     5 0.0135  Preprocessor1_Model1
## 2 brier_class     binary    0.136     5 0.00933 Preprocessor1_Model1
## 3 roc_auc        binary    0.612     5 0.0286  Preprocessor1_Model1

- report the average area under the ROC curve and the accuracy for all cross validation folds `collect`

The average area under the ROC curve is 0.612, while the accuracy
is 0.834.

10. Comment on which model performs better (one including wrkstat,
model 1, or the one excluding wrkstat, model 2) on the training
data based on area under the ROC curve.

```


Because model 1 has a greater area under the ROC curve, it performs better on the training data. Model one has an area of 0.614, while model 2 has an area of 0.612. This means that model 1 is a better predictor of the training data and better distinguishes between true and false values because it ranks positives closer to being true, and negatives closer to being false.

11. Fit both models to the testing data, plot the ROC curves for the predictions for both models, and calculate the areas under the ROC curve. Does your answer to the previous exercise hold for the testing data as well? Explain your reasoning. Note: If you haven't yet done so, you'll need to first train your workflows on the training data with the following, and then use these fit objects to calculate predictions for the test data.

```
gss16_fit_1 <- gss16_wflow_1 %>%
  fit(gss16_train)

gss16_fit_2 <- gss16_wflow_2 %>%
  fit(gss16_train)

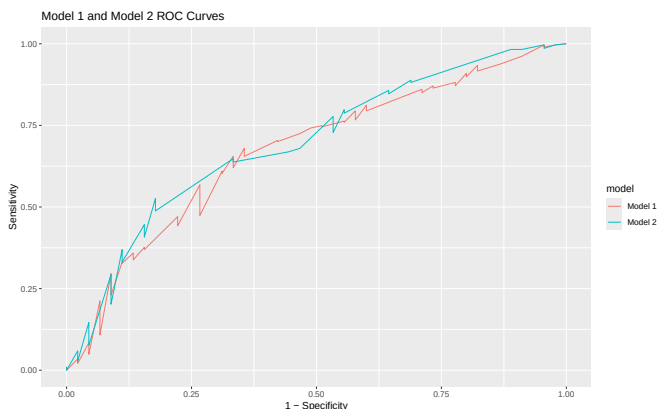
prediction_1 <- predict(gss16_fit_1, gss16_test, type = "prob") %>%
  bind_cols(gss16_test)

prediction_2 <- predict(gss16_fit_2, gss16_test, type = "prob") %>%
  bind_cols(gss16_test)

roc_1 <- prediction_1 %>%
  roc_curve(truth = advfront, .pred_Agree)

roc_2 <- prediction_2 %>%
  roc_curve(truth = advfront, .pred_Agree)

bind_rows(
  roc_1 %>% mutate(model = "Model 1"),
  roc_2 %>% mutate(model = "Model 2")
) %>%
ggplot(aes(x = 1 - specificity, y = sensitivity, color = model)) +
  geom_line() +
  labs(title = "Model 1 and Model 2 ROC Curves",
       x = "1 - Specificity", y = "Sensitivity")
```



```
roc_auc(prediction_1, truth = advfront, .pred_Agree)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 roc_auc binary      0.674
```

```
roc_auc(prediction_2, truth = advfront, .pred_Agree)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 roc_auc binary      0.696
```

No, the answer to the previous exercise does not hold for the testing data. On the training data, Model 1 had a higher area under curve for ROC, meaning that it was better at differentiating between positive and negative cases; however, for the test data, model 2 actually performs better than model 1 because it has an `roc_auc` of 0.696, while model 1 only has an `roc_auc` of 0.674.

Knit, commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards.

Harassment at work

In 2016, the GSS added a new question on harassment at work. The question is phrased as the following.

Over the past five years, have you been harassed by your superiors or co-workers at your job, for example, have you experienced any bullying, physical or psychological abuse?

Answers to this question are stored in the `harass5` variable in our dataset.

12. Create a subset of the data that only contains Yes and No answers for the harassment question. How many responses chose each of these answers?

```
gss16_harass5 <- gss16 %>%
  filter(harass5 %in% c("Yes", "No"))

gss16_harass5 %>%
  count(harass5)

## # A tibble: 2 x 2
##   harass5      n
##   <chr>    <int>
## 1 No      1136
## 2 Yes      237
```

1136 people voted no for the harassment question, while 237 people in the survey responded yes to the harassment question.

13. Describe how bootstrapping can be used to estimate the proportion of Americans who have been harassed by their superiors or co-workers at their job.

Bootstrapping could be used to estimate the proportion of Americans who have been harassed by their superiors or co-workers at their job by sampling the survey data with replacement multiple times.

Bootstrapping quantifies the uncertainty of the percentage of Americans harassed because it can be done multiple times and confidence intervals can be established to get a prediction of the actual percentage.

14. Calculate a 95% bootstrap confidence interval for the proportion of Americans who have been harassed by their superiors or co-workers at their job. Interpret this interval in context of the data.

Hint: Use the **infer** package workflow: `specify() %>% generate() %>% calculate() %>% get_confidence_interval()`

```
library(infer)
set.seed(1245)
boot_ci <- gss16_harass5 %>%
  specify(response = harass5, success = "Yes") %>%
  generate(reps = 1000, type = "bootstrap") %>%
  calculate(stat = "prop")
get_confidence_interval(boot_ci, level = 0.95)

## # A tibble: 1 x 2
```

```
##   lower_ci upper_ci
##      <dbl>   <dbl>
## 1    0.152    0.194
```

Interpretation: We are 95% confident that the proportion of Americans who have been harassed by their superiors or co-workers at their job is in between 15.2% and 19.4%.

15. Would you expect a 90% confidence interval to be wider or narrower than the interval you calculated above? Explain your reasoning.

I would expect a 90% confidence interval to be narrower than the interval I calculated above because to have a smaller confidence means to have a smaller range.

For example, if I was on a game show and had to guess the price of a car, I would have a higher confidence of being right if I said 50,000 to 60,000 dollars rather than 52,000 to 53,000 dollars because the range is bigger for 50,000 to 60,000.

Knit, *commit*, and *push your changes to GitHub with an appropriate commit message*. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards and review the md document on GitHub to make sure you're happy with the final state of your work.

Now go back through your write up to make sure you've answered all questions and all of your R chunks are properly labeled.

Once you decide that you are done with the homework, choose the knit drop down and select **Knit to tufte_handout** to generate a pdf. Download and submit that pdf to Canvas.

Common Errors and Troubleshooting

Data preparation errors:

- **Error: “Can’t subset columns that don’t exist”** → Check variable names in `select()` - they’re case-sensitive
- **Recoded variable still has old levels** → Make sure you saved changes back to the dataset with `<-`
- **Training/testing split doesn’t work** → Check that you ran `drop_na()` before `initial_split()`

Recipe errors:

- **Error: “All columns selected for the step should be numeric”** → Check that your outcome variable isn’t included in preprocessing steps (use `-all_outcomes()`)

- **Error: “object not found” in recipe** → Make sure all variables in the formula exist in your data
- **Recipe steps in wrong order** → Dummy variables should typically be created last

Workflow errors:

- **Error: “No method for objects of class recipe”** → Use `add_recipe()` not just the recipe name
- **Forgot to specify engine** → Make sure you used `set_engine("glm")` in your model specification

Cross-validation errors:

- **Error: “object not found”** → Set seed before creating folds; make sure you’re using `gss16_train` not `gss16_advfront`
- **Metrics are all NA** → Check that your outcome variable has correct factor levels

Testing data errors:

- **ROC curve won’t plot** → Make sure predictions include probability columns (use `type = "prob"`)
- **Error: “truth should be a factor”** → Your outcome variable needs to be a factor with matching levels in train and test

Bootstrap errors:

- **Error: “success level not found”** → Check spelling of success level (e.g., “Yes” is case-sensitive)
- **Confidence interval seems wrong** → Make sure you set a seed and used enough bootstrap resamples (1000+)